Upper Confidence Bound (UCB) Algorithm

A step-by-step, worked-example tutorial — Ad Click-Through Optimization

1. The Problem We're Solving

Imagine a company has **10 different ad designs** for the same product, but doesn't know which one gets the most clicks. Every time a new user visits the site, the company must pick *one* ad to show. After the user leaves, we find out whether they clicked (reward = 1) or not (reward = 0).

This is a classic **multi-armed bandit problem**: we want to show good-performing ads as much as possible (exploit), while still occasionally testing under-tried ads in case they're secretly better (explore). The **Upper Confidence Bound (UCB)** algorithm balances both automatically, round by round, using nothing more than counts and averages — no machine learning training required.

Our dataset (from your screenshot) is a table of 0s and 1s: each row is one round/user, each column (Ad 1 … Ad 10) records whether *that* ad *would have* been clicked, had it been shown. The algorithm only ever gets to see the value for the ad it actually chooses.

2. The Algorithm, In Three Steps

Step 1 — Keep two running numbers per ad. For every ad i, track $N_i(n)$: how many times it's been shown up to round n, and $R_i(n)$: the sum of rewards (clicks) it has earned up to round n.

Step 2 — Turn those into an average and a confidence margin. The average reward estimates how good the ad looks so far. The margin shrinks the more the ad has been tested, and grows with the round number n — capturing "how sure are we?"

$$\text{average reward: } \bar{r}_i(n) = R_i(n) / N_i(n)$$

$$\text{confidence margin: } \Delta_i(n) = \sqrt{(1.5 \times \ln(n) / N_i(n))}$$

Step 3 — Pick the optimist's choice. Compute the Upper Confidence Bound $UCB_i = \bar{r}_i(n) + \Delta_i(n)$ for every ad, and show the ad with the **highest** UCB. This is "optimism in the face of uncertainty" — an ad we're unsure about gets the benefit of the doubt via a big Δ, so it still gets a chance.

The bootstrap trick: if an ad has never been shown yet, $N_i(n)=0$, so $\Delta_i(n)$ is technically infinite. That single fact automatically forces the algorithm to try every ad at least once before it starts trusting averages — no special-case code needed!

3. Phase 1 (Rounds 1–10): Forced Exploration

Because every ad starts with $N_i=0$ (an infinite UCB), the algorithm simply works through the ads in order, one per round, collecting its very first data point for each:

Round n	Ad shown	Reward observed	N after (all 10 ads)	R after (all 10 ads)
1	**Ad 1**	1	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
2	**Ad 2**	0	[1, 1, 0, 0, 0, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
3	**Ad 3**	0	[1, 1, 1, 0, 0, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
4	**Ad 4**	0	[1, 1, 1, 1, 0, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
5	**Ad 5**	0	[1, 1, 1, 1, 1, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
6	**Ad 6**	0	[1, 1, 1, 1, 1, 1, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
7	**Ad 7**	0	[1, 1, 1, 1, 1, 1, 1, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
8	**Ad 8**	0	[1, 1, 1, 1, 1, 1, 1, 1, 0, 0]	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
9	**Ad 9**	0	[1, 1, 1, 1, 1, 1, 1, 1, 1, 0]	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]

10	**Ad 10**	0	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1]	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]

After round 10, every ad has been tried exactly once. Ad 1 got lucky and earned a reward of 1; every other ad returned 0 the one time it was tried. Real competition between ads can now begin.

4. Phase 2: Worked Iterations With the Real UCB Formula

From round 11 onward every ad has $N_i > 0$, so we compute the full formula each time. Let's do this by hand for five rounds — the first three below, plus **two additional iterations** (rounds 14 and 15) to really see the pattern settle in.

Round n = 11

Ad i	Ní(n)	Rí(n)	r̄í(n)	Δí(n)	UCB = r̄+Δ
Ad 1	1	1	1.0000	1.8965	**2.8965**
Ad 2	1	0	0.0000	1.8965	**1.8965**
Ad 3	1	0	0.0000	1.8965	**1.8965**
Ad 4	1	0	0.0000	1.8965	**1.8965**
Ad 5	1	0	0.0000	1.8965	**1.8965**
Ad 6	1	0	0.0000	1.8965	**1.8965**
Ad 7	1	0	0.0000	1.8965	**1.8965**
Ad 8	1	0	0.0000	1.8965	**1.8965**
Ad 9	1	0	0.0000	1.8965	**1.8965**
Ad 10	1	0	0.0000	1.8965	**1.8965**

Highest UCB → **Ad 1** is shown to the user. Actual outcome in the dataset: **reward = 0** (no click). Updated counts: $N_1 = 2$, $R_1 = 1$, running total reward = 1.

Round n = 12

Ad i	Ní(n)	Rí(n)	r̄í(n)	Δí(n)	UCB = r̄+Δ
Ad 1	2	1	0.5000	1.3652	**1.8652**
Ad 2	1	0	0.0000	1.9306	**1.9306**
Ad 3	1	0	0.0000	1.9306	**1.9306**
Ad 4	1	0	0.0000	1.9306	**1.9306**
Ad 5	1	0	0.0000	1.9306	**1.9306**
Ad 6	1	0	0.0000	1.9306	**1.9306**
Ad 7	1	0	0.0000	1.9306	**1.9306**
Ad 8	1	0	0.0000	1.9306	**1.9306**
Ad 9	1	0	0.0000	1.9306	**1.9306**
Ad 10	1	0	0.0000	1.9306	**1.9306**

Highest UCB → **Ad 2** is shown to the user. Actual outcome in the dataset: **reward = 0** (no click). Updated counts: $N_2 = 2$, $R_2 = 0$, running total reward = 1.

Round n = 13

Ad i	Ní(n)	Rí(n)	r̄í(n)	Δí(n)	UCB = r̄+Δ
Ad 1	2	1	0.5000	1.3870	**1.8870**

Ad 2	2	0	0.0000	1.3870	**1.3870**
Ad 3	1	0	0.0000	1.9615	**1.9615**
Ad 4	1	0	0.0000	1.9615	**1.9615**
Ad 5	1	0	0.0000	1.9615	**1.9615**
Ad 6	1	0	0.0000	1.9615	**1.9615**
Ad 7	1	0	0.0000	1.9615	**1.9615**
Ad 8	1	0	0.0000	1.9615	**1.9615**
Ad 9	1	0	0.0000	1.9615	**1.9615**
Ad 10	1	0	0.0000	1.9615	**1.9615**

Highest UCB → **Ad 3** is shown to the user. Actual outcome in the dataset: **reward = 0** (no click). Updated counts: $N_3 = 2$, $R_3 = 0$, running total reward = 1.

Round n = 14 extra iteration #1

Ad i	Ní(n)	Rí(n)	r̄í(n)	Δí(n)	UCB = r̄+Δ
Ad 1	2	1	0.5000	1.4069	**1.9069**
Ad 2	2	0	0.0000	1.4069	**1.4069**
Ad 3	2	0	0.0000	1.4069	**1.4069**
Ad 4	1	0	0.0000	1.9896	**1.9896**
Ad 5	1	0	0.0000	1.9896	**1.9896**
Ad 6	1	0	0.0000	1.9896	**1.9896**
Ad 7	1	0	0.0000	1.9896	**1.9896**
Ad 8	1	0	0.0000	1.9896	**1.9896**
Ad 9	1	0	0.0000	1.9896	**1.9896**
Ad 10	1	0	0.0000	1.9896	**1.9896**

Highest UCB → **Ad 4** is shown to the user. Actual outcome in the dataset: **reward = 0** (no click). Updated counts: $N_4 = 2$, $R_4 = 0$, running total reward = 1.

Round n = 15 extra iteration #2

Ad i	Ní(n)	Rí(n)	r̄í(n)	Δí(n)	UCB = r̄+Δ
Ad 1	2	1	0.5000	1.4251	**1.9251**
Ad 2	2	0	0.0000	1.4251	**1.4251**
Ad 3	2	0	0.0000	1.4251	**1.4251**
Ad 4	2	0	0.0000	1.4251	**1.4251**
Ad 5	1	0	0.0000	2.0155	**2.0155**
Ad 6	1	0	0.0000	2.0155	**2.0155**
Ad 7	1	0	0.0000	2.0155	**2.0155**
Ad 8	1	0	0.0000	2.0155	**2.0155**
Ad 9	1	0	0.0000	2.0155	**2.0155**
Ad 10	1	0	0.0000	2.0155	**2.0155**

Highest UCB → **Ad 5** is shown to the user. Actual outcome in the dataset: **reward = 0** (no click). Updated counts: $N_5 = 2$, $R_5 = 0$, running total reward = 1.

What's happening here? Ad 1's average reward is stuck at a modest 0.5 (it clicked once out of two tries), so its confidence margin shrinks fast as its own N grows. Meanwhile every other ad still has an average of exactly 0 — but a *larger* Δ because it's been tried less. The algorithm keeps rotating through the "least certain" ad, because right now uncertainty (exploration) outweighs the small edge Ad 1 has shown (exploitation). This is UCB doing its job: nothing looks clearly best yet, so it keeps sampling fairly.

5. Rounds 16–20: The Pattern Continues

Running the exact same three steps for the remaining rounds produces this sequence. Notice it's the same round-robin order all over again — every "zero-average" ad ties on average reward, so whichever has been tried the least (smallest N, hence the biggest Δ) wins each time:

Round n	Ad shown	Reward observed	Running total reward
16	**Ad 6**	0	1
17	**Ad 7**	0	1
18	**Ad 8**	0	1
19	**Ad 9**	0	1
20	**Ad 10**	0	1

6. Visualizing the 20 Rounds

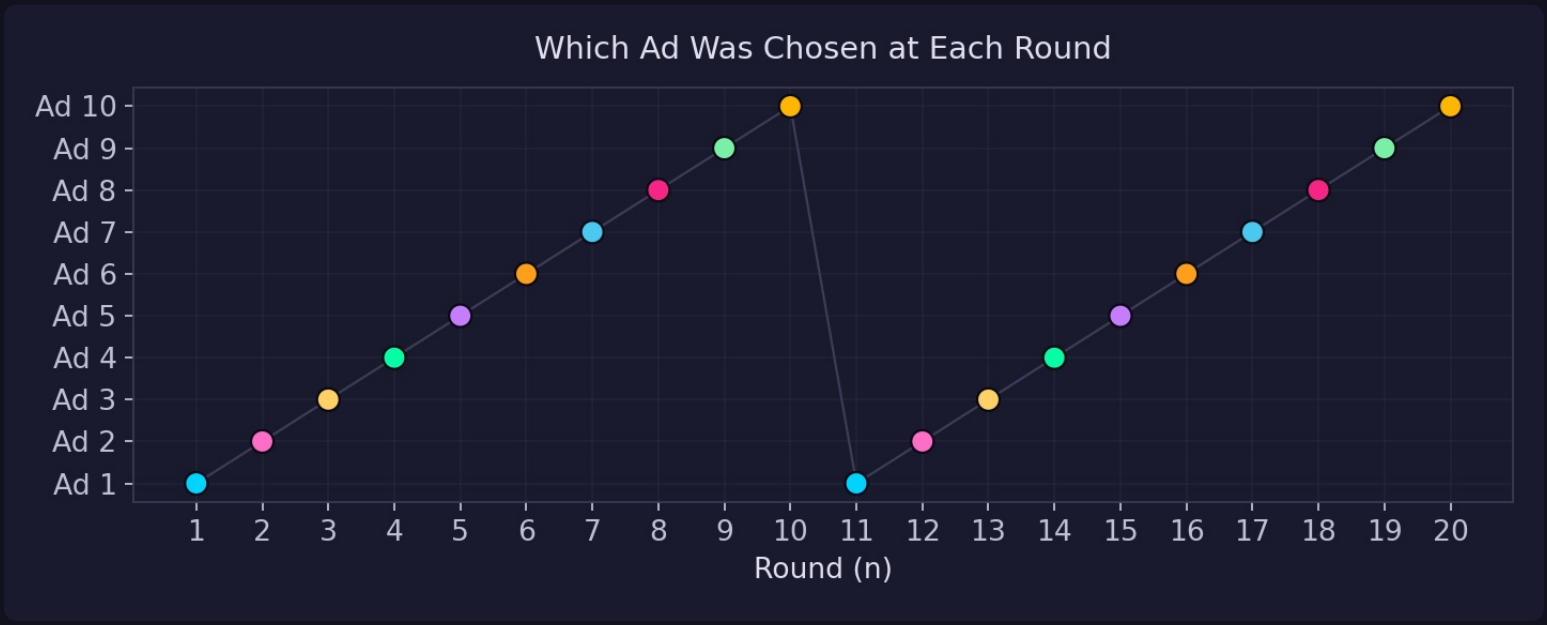


Fig. 1 — The sequence of ads chosen. Notice the perfect round-robin: Ads 1→10, then 1→10 again, because no ad has separated itself from the pack yet.

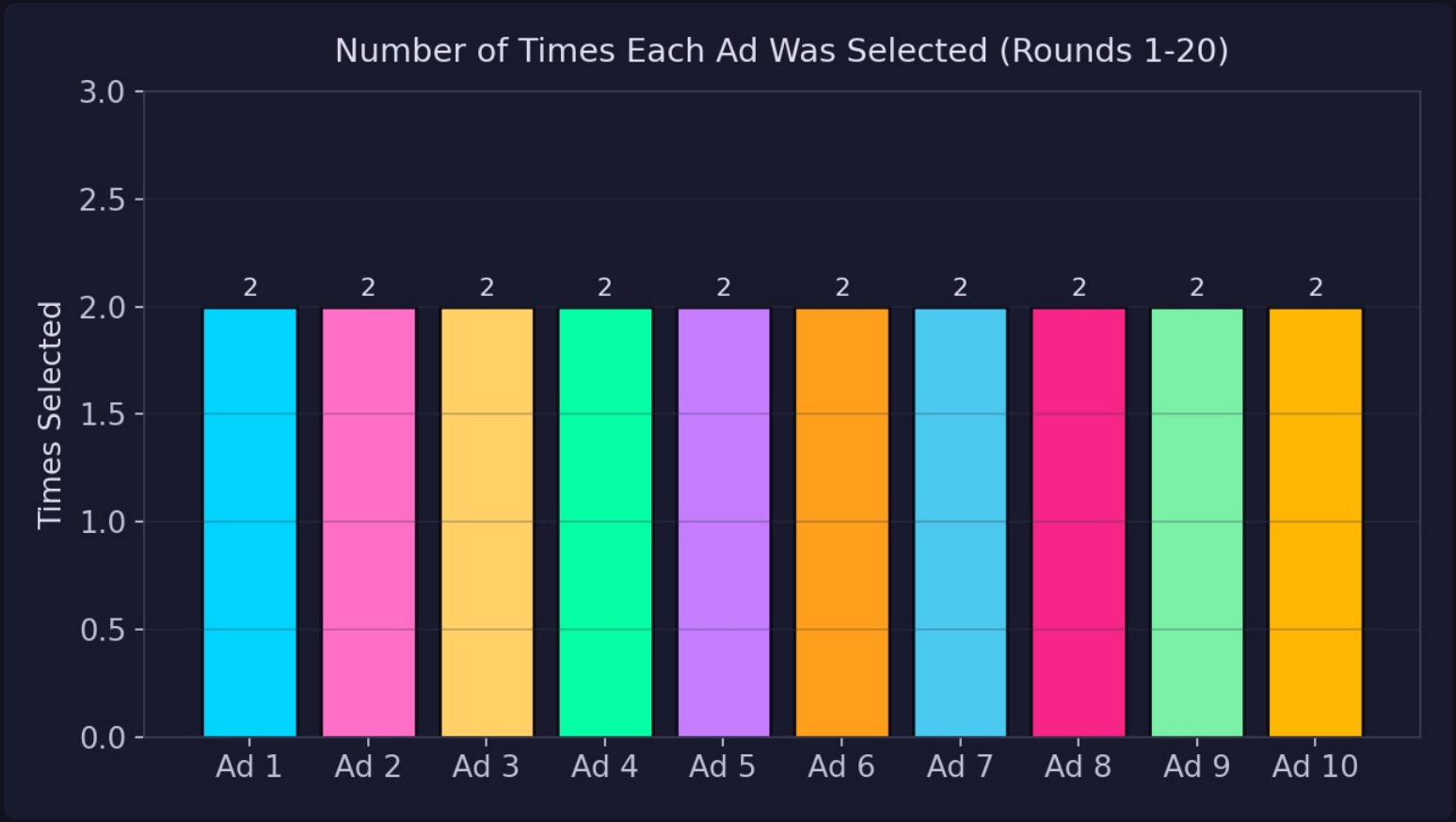


Fig. 2 — After 20 rounds every ad has been tried exactly twice — a direct consequence of the tie-breaking behavior we walked through above.

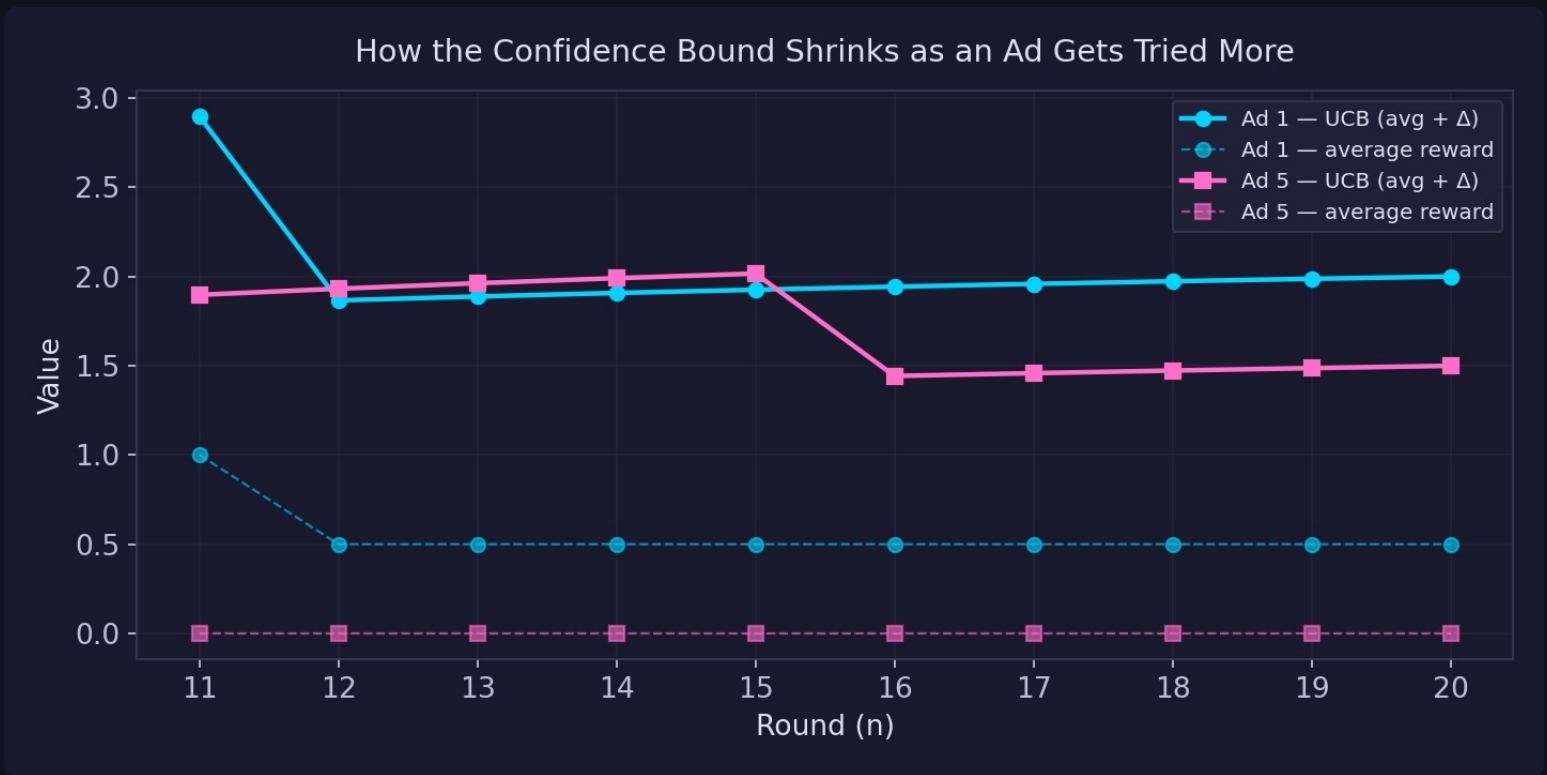


Fig. 3 — Ad 1's UCB (solid cyan) starts high thanks to its early click, then cools toward its true average (dashed) as Δ shrinks. Ad 5's UCB (pink) briefly overtakes it once Ad 5 has been tried less recently — then falls back once Ad 5 is also sampled again. This tug-of-war between "known good" and "under-tested" ads is the entire idea behind UCB.

7. Key Takeaways

- **No probability distributions or training needed** — UCB only needs two counters (N and R) per option.
- **Exploration is automatic and self-limiting**: $\Delta_i(n)$ shrinks as $1/\sqrt{N_i}$, so a heavily-tested ad's uncertainty fades, while $\ln(n)$ grows so slowly that exploration never fully stops — just enough to keep double-checking losers occasionally.
- **With only 20 rounds and every "losing" ad tied at 0 average**, we mostly saw exploration in action. Given many more rounds, Ad 1

(or whichever ad truly clicks more often) would pull ahead in average reward by enough that its UCB stays on top, and the algorithm would show it far more often — converging toward the best ad.
- This is exactly the mechanism used in the classic "Ads C.T.R. Optimisation" exercise: over 10,000 simulated rounds, UCB reliably discovers which ad has the highest true click-through rate.